

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO CÁC HỆ THỐNG PHÂN TÁN

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG BÁN HÀNG TRỰC TUYẾN PHÂN TÁN
THEO KIẾN TRÚC MICROSERVICES**

Giảng viên : Kim Ngọc Bách

Nhóm BTL 9 – Nhóm lớp 2

Tên Thành Viên

Mã Sinh Viên

Nguyễn Thị Như Quỳnh

: B22DCCN679

Hồ Ngọc Linh

: D22DCCN488

Khổng Minh Hiếu

: D22DCCN308

Hà Nội, tháng 05 năm 2026

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Giải thích
ACK	Acknowledgement	Tín hiệu xác nhận
AMQP	Advanced Message Queuing Protocol	Giao thức hàng đợi thông điệp nâng cao — giao thức tầng ứng dụng mà RabbitMQ sử dụng.
API	Application Programming Interface	Giao diện lập trình ứng dụng
BCrypt		Thuật toán băm mật khẩu một chiều có độ bảo mật cao, tích hợp cơ chế salt tự động chống tấn công vét cạn.
CI/CD	Continuous Integration / Continuous Delivery	Tích hợp liên tục và Phân phối liên tục.
DB	Database	Cơ sở dữ liệu.
DLQ	Dead Letter Queue	Hàng đợi thư chết - nơi lưu trữ các tin nhắn (message) xử lý thất bại sau số lần thử lại (retry) tối đa.
Docker		Nền tảng đóng gói (container hóa) ứng dụng, giúp chạy phần mềm độc lập cùng môi trường nhất quán trên mọi hệ thống.

E-Commerce	Electronic Commerce	Thương mại điện tử - hoạt động mua bán hàng hóa, dịch vụ thông qua mạng Internet.
EDA	Event-Driven Architecture	Kiến trúc hướng sự kiện
GCP	Google Cloud Platform	Nền tảng hạ tầng điện toán đám mây được cung cấp bởi Google.
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
JPA	Java Persistence API	Đặc tả Java cho việc ánh xạ đối tượng sang cơ sở dữ liệu quan hệ (ORM).
JSON	JavaScript Object Notation	Định dạng dữ liệu nhẹ, dễ đọc, được sử dụng phổ biến để trao đổi dữ liệu giữa các service.
JWT	JSON Web Token	Chuỗi mã hóa mang thông tin định danh và phân quyền người dùng, được ký số để đảm bảo tính toàn vẹn.
LB	Load Balancer	Bộ cân bằng tải
MQ	Message Queue	Hàng đợi thông điệp
NACK	Negative Acknowledgement	Tín hiệu từ chối

ORM	Object-Relational Mapping	Kỹ thuật ánh xạ dữ liệu giữa đối tượng trong ngôn ngữ lập trình và bảng trong cơ sở dữ liệu quan hệ.
PENDING		Trạng thái đơn hàng đang chờ xử lý trên hệ thống.
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
REST	Representational State Transfer	Kiến trúc thiết kế API dựa trên giao thức HTTP, sử dụng các phương thức GET, POST, PUT, DELETE để thao tác tài nguyên.
SLF4J	Simple Logging Facade for Java	Thư viện giao diện nhật ký (logging) thống nhất cho ngôn ngữ Java.
SMTP	Simple Mail Transfer Protocol	Giao thức truyền tải thư điện tử đơn giản
SPA	Single Page Application	Ứng dụng web một trang - tải một lần và cập nhật nội dung động mà không cần tải lại toàn bộ trang.
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc dùng để thao tác với cơ sở dữ liệu quan hệ.
UI	User Interface	Giao diện người dùng.

URI	Uniform Resource Identifier	Chuỗi ký tự dùng để định danh tài nguyên trên mạng.
URL	Uniform Resource Locator	Địa chỉ định vị tài nguyên trên Internet.
VM	Virtual Machine	Máy ảo

DANH MỤC HÌNH ẢNH

Hình ảnh 2.1 : tổng quan kiến trúc hệ thống	12
Hình ảnh 5.1 : tiến trình xử lý luồng nghiệp vụ	23
Hình ảnh 6.1 : Giao diện trang chủ	28
Hình ảnh 6.2 : Giao diện đăng nhập	29
Hình ảnh 6.3 : giao diện sản phẩm	30
Hình ảnh 6.4 : giao diện hiển thị thanh toán	30
Hình ảnh 6.5 : Giao diện chi tiết đơn hàng.....	32
Hình ảnh 6.6 : Lịch sử đơn hàng	32
Hình ảnh 6.7 : Giao diện trang quản trị hệ thống	33
Hình ảnh 6.8 : Giao diện giám sát đơn hàng.....	34
Hình ảnh 6.9 : Giao diện quản lý sản phẩm	35
Hình ảnh 6.10 : Giao diện quản lý tài khoản	35

MỤC LỤC

LỜI CẢM ƠN.....	8
CHƯƠNG 1: TỔNG QUAN VÀ MÔ TẢ BÀI TOÁN	9
1.1. Tổng quan bối cảnh và lý do chọn đề tài	9
1.2. Phân tích các chức năng chính	9
1.3. Đối tượng sử dụng.....	10
CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG.....	12
2.1. Tổng quan kiến trúc.....	12
2.2. Công nghệ sử dụng (Tech Stack).....	14
CHƯƠNG 3: MÔ TẢ CHI TIẾT CÁC MICROSERVICES	17
3.1. User Service (Cổng 8081)	17
3.2. Product Service (Cổng 8082)	17
3.3. Order Service (Cổng 8083).....	18
3.4. Notification Service (Không mở cổng).....	18
CHƯƠNG 4: THIẾT KẾ RESTFUL API.....	19
4.1. User Service API	19
4.2. Product Service API	20
4.3. Order Service API.....	21
CHƯƠNG 5: LƯỒNG MESSAGE QUEUE	23
5.1. Tổng quan cơ chế hướng sự kiện (Event-Driven Architecture).....	23
5.2. Đặc tả chi tiết tiến trình xử lý luồng nghiệp vụ đặt hàng (Sequence Workflow).....	23
5.3. Thông số cấu hình kỹ thuật hạ tầng RabbitMQ.....	24

CHƯƠNG 6 : THỬ NGHIỆM VÀ ĐÁNH GIÁ.....	27
6.1. Môi trường thử nghiệm	27
6.2. Kịch bản thử nghiệm	28
6.3. Đánh giá tiêu chí.....	37
6.4. Nhận xét ưu điểm và hạn chế.....	39
6.4.1. Ưu điểm.....	39
6.4.2. Hạn chế.....	40
TÀI LIỆU THAM KHẢO	42

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy Kim Ngọc Bách - giảng viên môn Các Hệ thống Phân tán tại Học viện Công nghệ Bưu chính Viễn thông (PTIT).

Trong suốt quá trình học tập môn học, Thầy đã không chỉ truyền đạt những kiến thức lý thuyết nền tảng về kiến trúc hệ thống phân tán, mà còn định hướng tư duy thiết kế hệ thống thực tế một cách bài bản và có chiều sâu. Những bài giảng của Thầy về các mô hình giao tiếp liên service, cơ chế đảm bảo tính nhất quán dữ liệu trong môi trường phân tán, cũng như các nguyên tắc xây dựng hệ thống chịu lỗi đã trang bị cho nhóm nền tảng vững chắc để tiếp cận và hoàn thành bài tập lớn này.

Bài tập lớn về Phát triển ứng dụng Microservices là một cơ hội thực hành quý giá mà Thầy đã thiết kế và giao cho nhóm. Thông qua quá trình xây dựng hệ thống bán hàng trực tuyến phân tán — từ việc phân tách dịch vụ theo nguyên tắc Bounded Context, thiết kế RESTful API, tích hợp Message Queue với RabbitMQ cho đến triển khai toàn bộ hệ thống bằng Docker Compose — nhóm đã được trải nghiệm trực tiếp những thách thức và giá trị thực tiễn của kiến trúc Microservices mà không một bài học lý thuyết nào có thể thay thế được.

Nhóm cũng xin chân thành cảm ơn Thầy đã dành thời gian hướng dẫn, giải đáp thắc mắc và hỗ trợ nhóm trong suốt quá trình thực hiện đồ án. Sự tận tâm và nhiệt huyết của Thầy trong giảng dạy là nguồn động lực lớn giúp nhóm không ngừng cố gắng hoàn thiện sản phẩm.

Mặc dù nhóm đã cố gắng hết sức trong quá trình nghiên cứu và triển khai, song với giới hạn về kiến thức và kinh nghiệm thực tế, báo cáo và sản phẩm chắc chắn còn nhiều thiếu sót. Nhóm rất mong nhận được những nhận xét, góp ý quý báu từ Thầy để nhóm có thể học hỏi, rút kinh nghiệm và hoàn thiện hơn trong các dự án tiếp theo.

Nhóm xin chân thành cảm ơn!

CHƯƠNG 1: TỔNG QUAN VÀ MÔ TẢ BÀI TOÁN

1.1. Tổng quan bối cảnh và lý do chọn đề tài

Trong bối cảnh nền kinh tế số phát triển bùng nổ, các hệ thống Thương mại điện tử (E-Commerce) không còn đơn thuần là những trang web hiển thị sản phẩm và lưu đơn hàng cơ bản. Các hệ thống hiện đại phải đối mặt với những thách thức cực kỳ lớn về mặt hạ tầng công nghệ: lượng truy cập tăng đột biến vào các khung giờ vàng (Flash Sale), các chiến dịch quảng cáo quy mô lớn thu hút hàng triệu người dùng cùng lúc, và yêu cầu khắt khe về trải nghiệm người dùng (độ trễ thấp, tính sẵn sàng 24/7).

Theo kiến trúc truyền thống (Monolith - Hệ thống nguyên khối), toàn bộ các module chức năng như Quản lý người dùng, Sản phẩm, Đặt hàng, Kho bãi và Thông báo được đóng gói chung vào một mã nguồn duy nhất và chạy trên cùng một máy chủ. Mô hình này bộc lộ những hạn chế chí mạng khi hệ thống mở rộng:

- Khả năng mở rộng kém (Scalability): Khi lượng người dùng vào xem sản phẩm tăng gấp 100 lần, hệ thống bắt buộc phải nhân bản toàn bộ khối ứng dụng (gồm cả các module không bị quá tải như Nhận thông báo, Quản lý tài khoản), gây lãng phí tài nguyên máy chủ một cách vô ích.
- Tính chịu lỗi thấp (Fault Tolerance): Do các module chia sẻ chung bộ nhớ và tiến trình, một lỗi logic nhỏ xảy ra ở module gửi email thông báo (ví dụ: máy chủ mail bị nghẽn) có thể làm tràn bộ nhớ (Out of Memory) và kéo sập toàn bộ hệ thống, khiến khách hàng không thể đăng nhập hoặc đặt hàng.
- Rào cản công nghệ và triển khai (Deployment): Mỗi lần cập nhật một tính năng nhỏ cho module Sản phẩm, lập trình viên vẫn phải build và triển khai lại toàn bộ hệ thống, tăng rủi ro gián đoạn dịch vụ (Downtime).

Để giải quyết triệt để các bài toán trên, nhóm quyết định nghiên cứu và xây dựng Hệ thống bán hàng trực tuyến phân tán dựa trên Kiến trúc Microservices. Bằng cách phân rã hệ thống thành các dịch vụ độc lập (Decomposed Services), mỗi dịch vụ chỉ chịu trách nhiệm cho một miền chức năng duy nhất (Bounded Context). Sự kết hợp giữa giao tiếp đồng bộ qua RESTful API và bất đồng bộ qua Message Queue (RabbitMQ) giúp tối ưu hóa hiệu năng, tăng cường tính chịu lỗi và đảm bảo tính nhất quán dữ liệu trong một hệ thống phân tán phức tạp.

1.2. Phân tích các chức năng chính

Hệ thống được thiết kế tập trung vào 5 chức năng chính, giải quyết trọn vẹn vòng đời của một giao dịch thương mại điện tử:

- **Quản lý tài khoản:** Khởi tạo tài khoản người dùng mới và thiết lập cơ chế đăng nhập an toàn. Để đảm bảo an toàn thông tin, hệ thống áp dụng thuật toán mã hóa một chiều BCrypt cho mật khẩu. Khi xác thực thành công, hệ thống cấp phát một mã cấu trúc dạng JSON Web Token (JWT) chứa thông tin định danh và vai trò (Role-based access control) của người dùng. Token này đóng vai trò như một "chứng minh thư kỹ thuật số" để người dùng truy cập hợp pháp vào các dịch vụ khác thông qua API Gateway.
- **Quản lý sản phẩm :** Quản lý toàn bộ thông tin phi cấu trúc và cấu trúc của hàng hóa bao gồm: tên, mô tả, hình ảnh, giá cả và phân loại mục hàng. Phân hệ này cung cấp các luồng truy vấn dữ liệu hiệu năng cao cho khách hàng, đồng thời cung cấp các cổng kiểm tra nhanh dữ liệu tồn kho dành riêng cho các dịch vụ nội bộ hệ thống.
- **Xử lý đơn hàng:** Đây là trung tâm điều phối của hệ thống, tiếp nhận yêu cầu mua hàng từ khách hàng. Nhiệm vụ là kiểm tra tính hợp lệ của đơn hàng (thông tin người mua, tính hợp pháp của sản phẩm), ghi nhận trạng thái đơn hàng vào cơ sở dữ liệu và kích hoạt các quy trình xử lý sau đặt hàng (trừ kho, gửi mail).
- **Quản lý tồn kho tự động:** Giám sát chặt chẽ số lượng hàng hóa vật lý còn lại trong kho. Hoạt động như một consumer ngầm, liên tục cập nhật tăng/giảm số lượng tồn kho dựa trên các sự kiện đơn hàng được phát động từ hệ thống, ngăn chặn tình trạng bán quá số lượng thực tế (Overselling).
- **Thông báo tự động :**Chịu trách nhiệm tương tác và chăm sóc khách hàng bằng cách gửi email xác nhận thông tin đơn hàng, hóa đơn điện tử hoặc mã giảm giá. Tiến trình này được tách rời hoàn toàn khỏi luồng mua hàng chính để đảm bảo tốc độ phản hồi tối đa cho người dùng.

1.3.Đối tượng sử dụng

Hệ thống phân định rõ ràng hai nhóm đối tượng tác động với các quyền hạn và hành vi được kiểm soát nghiêm ngặt:

- **Khách hàng (Customer - Người dùng cuối):** Duyệt danh sách hàng hóa, tìm kiếm sản phẩm theo nhu cầu, xem thông tin chi tiết cấu hình/giá cả. Tiến hành thêm sản phẩm vào giỏ hàng và thực hiện lệnh đặt hàng (Checkout). Sau khi mua hàng, có quyền truy cập vào phân hệ lịch sử để theo dõi tiến trình xử lý đơn (Chờ duyệt, Đang giao, Đã giao).

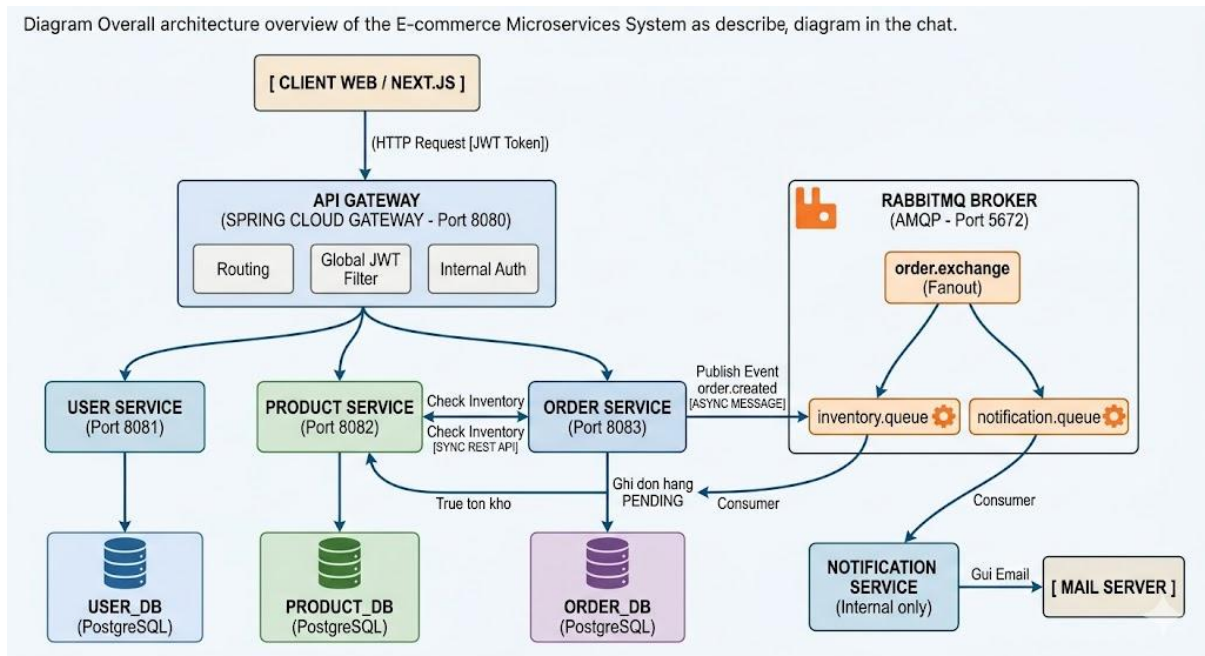
Chỉ được phép xem thông tin công khai và chỉnh sửa/truy cập dữ liệu cá nhân của chính mình dựa trên Token JWT được cấp.

- Quản trị viên (Admin - Nhà quản lý): Vận hành hệ thống bằng cách cập nhật lượng hàng hóa (Thêm sản phẩm mới, điều chỉnh giá bán, sửa đổi mô tả hoặc xóa ản các sản phẩm ngừng kinh doanh). Giám sát luồng vận hành của toàn bộ đơn hàng phát sinh từ tất cả các khách hàng trên hệ thống để chuyển trạng thái xử lý. Mọi API tác động đến cấu trúc dữ liệu sản phẩm hoặc thay đổi trạng thái đơn hàng của hệ thống đều cấu hình một bộ lọc (Filter) tại API Gateway để kiểm tra quyền Admin trong Token JWT, ngăn chặn hoàn toàn các hành vi giả mạo request từ bên ngoài.

CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG

2.1. Tổng quan kiến trúc

Hệ thống áp dụng mô hình kết hợp giữa Kiến trúc hướng sự kiện (Event-Driven Architecture), mẫu thiết kế API Gateway và mô hình Mỗi dịch vụ một cơ sở dữ liệu (Database-per-service) nhằm bẻ gãy tính phụ thuộc chặt chẽ giữa các thành phần.



Hình ảnh 2.1: tổng quan kiến trúc hệ thống

Dựa trên sơ đồ kiến trúc tổng quan, hệ thống bán hàng trực tuyến phân tán được cấu trúc thành 4 tầng logic độc lập nhằm tối ưu hóa hiệu năng, tăng cường tính chịu lỗi và đảm bảo an toàn thông tin dữ liệu:

a) Tầng Giao diện Người dùng (Client Side Layer)

Ứng dụng Single Page Application (SPA) xây dựng trên nền tảng Next.js (chạy tại cổng 3000).

Nguyên lý vận hành: Đây là lớp giao tiếp trực tiếp với người dùng cuối (Khách hàng và Quản trị viên). Nhằm đảm bảo an ninh hạ tầng, Client hoàn toàn bị cô lập và không có kiến thức về cấu trúc mạng nội bộ, địa chỉ IP hay cổng dịch vụ (Port) của các Microservices phía sau. Mọi tương tác, truy vấn dữ liệu từ Client bắt buộc phải đóng gói dưới dạng các giao thức HTTP Request và gửi tập trung về một đầu mối duy nhất là API Gateway. Đối với các tác vụ yêu cầu định danh,

Client có trách nhiệm đính kèm chuỗi mã hóa JSON Web Token (JWT) vào tiêu đề (Header) của gói tin.

b) Tầng Cửa ngõ Hệ thống (API Gateway Layer)

Spring Cloud Gateway (chạy tại cổng 8080).

Nguyên lý vận hành: Gateway hoạt động như một lá chắn bảo mật và là bộ điều phối giao thông trung tâm của toàn hệ thống thông qua hai cơ chế cốt lõi:

- **Xác thực tập trung (Global JWT Authentication Filter):** Khi tiếp nhận các request thuộc diện bảo mật (ví dụ: đặt hàng, xem lịch sử giao dịch), Gateway sẽ chặn tiến trình và thực hiện một cuộc gọi REST API nội bộ sang đầu cổng `/internal/auth/validate` của User Service để kiểm tra tính toàn vẹn của chữ ký số và thời hạn sống của Token JWT. Nếu Token không hợp lệ, hệ thống lập tức từ chối và phản hồi mã lỗi 401 Unauthorized.
- **Định tuyến động (Dynamic Routing):** Khi request được xác thực thành công, Gateway sẽ phân tích cấu trúc URI (Uniform Resource Identifier) để định tuyến chính xác gói tin xuống các microservice đích nằm trong mạng nội bộ (ví dụ: các request có tiền tố `/api/v1/products/*` được chuyển tiếp đến Product Service, `/api/v1/orders/*` được chuyển tiếp đến Order Service).

c) Tầng Các Dịch vụ Nghiệp vụ Lõi (Core Microservices Layer)

Các dịch vụ thành phần được bẻ gãy tính phụ thuộc (Decoupling) theo mẫu thiết kế Database-per-service. Mỗi dịch vụ thực thi trong một tiến trình độc lập và toàn quyền sở hữu một cơ sở dữ liệu quan hệ (PostgreSQL) logic riêng biệt:

User Service (Cổng 8081 - Database user_db): Quản lý thực thể người dùng, thực thi thuật toán băm một chiều BCrypt để bảo mật mật khẩu khi đăng ký và chịu trách nhiệm sinh chuỗi mã JWT khi người dùng đăng nhập thành công.

Product Service (Cổng 8082 - Database product_db): Quản lý danh mục hàng hóa, giá cả và số lượng tồn kho vật lý của sản phẩm. Dịch vụ này cung cấp cả hai cổng giao tiếp: cổng đồng bộ phục vụ truy vấn kho và cổng bất đồng bộ phục vụ cập nhật số dư kho.

Order Service (Cổng 8083 - Database order_db): Hạt nhân điều phối toàn bộ vòng đời của một đơn hàng (từ trạng thái khởi tạo PENDING cho đến khi hoàn thành).

d) Tầng Hạ tầng Tích hợp và Tương tác Liên dịch vụ (Integration Layer)

Để giải quyết bài toán hiệu năng và tính nhất quán dữ liệu (Data Consistency) trong môi trường phân tán, hệ thống áp dụng mô hình giao tiếp lai (Hybrid Communication):

Giao tiếp Đồng bộ (Synchronous Communication via REST API): Khi khách hàng thực hiện lệnh Checkout, Order Service bắt buộc phải xác thực tính sẵn sàng của hàng hóa theo thời gian thực nhằm ngăn chặn lỗi bán quá số lượng (Overselling). Order Service sẽ sử dụng REST Client để thực hiện một cuộc gọi chặn (Blocking Call) trực tiếp sang Product Service. Nếu sản phẩm hợp lệ và đủ số lượng, Order Service mới tiến hành ghi nhận bản ghi đơn hàng với trạng thái PENDING vào cơ sở dữ liệu order_db, đồng thời lập tức giải phóng Client bằng mã phản hồi 201 Created.

Giao tiếp Bất đồng bộ (Asynchronous Event-Driven via RabbitMQ): Ngay sau khi giải phóng Client, Order Service (đóng vai trò Producer) sẽ phát hành một sự kiện mang tên order.created chứa toàn bộ metadata của đơn hàng vào RabbitMQ Fanout Exchange (Cổng 5672). Theo thuật toán mẫu của Fanout, Exchange này sẽ nhân bản thông điệp thành các bản sao độc lập và đẩy đồng thời vào hai hàng đợi:

1. inventory.queue: Product Service (lúc này đóng vai trò Consumer) tự động nhận thông điệp từ hàng đợi về để thực thi tác vụ trừ số lượng tồn kho vật lý trong product_db.
2. notification.queue: Notification Service (một dịch vụ chạy ẩn nội bộ, không mở cổng mạng ra bên ngoài) tiêu thụ thông điệp để thực hiện quy trình kết nối với Mail Server bên ngoài, gửi email hóa đơn điện tử cho khách hàng một cách bất đồng bộ.

Cơ chế hướng sự kiện này cam kết rằng, ngay cả khi Notification Service hoặc Mail Server gặp sự cố ngừng hoạt động (Downtime), thông điệp đơn hàng vẫn được lưu trữ an toàn, bền vững (Durable) trên đĩa cứng của RabbitMQ Broker và sẽ được xử lý bù tự động ngay khi dịch vụ thông báo được khôi phục, đảm bảo tính chịu lỗi tuyệt đối (Fault Tolerance) cho toàn hệ thống.

2.2. Công nghệ sử dụng (Tech Stack)

Toàn bộ hệ thống Backend được xây dựng trên hệ sinh thái mạnh mẽ của Java Spring Cloud, kết hợp với các giải pháp lưu trữ và điều phối thông điệp chuẩn công nghiệp.

Thành phần	Công nghệ sử dụng	Vai trò trong hệ thống	Cổng kết nối (Port)
Frontend	Next.js (TypeScript)	Xây dựng giao diện tương tác người dùng phía Client.	3000
API Gateway	Spring Cloud Gateway	Điểm tiếp nhận request tập trung, thực hiện định tuyến và kiểm tra token JWT.	8080
User Service	Spring Boot + Spring Data JPA	Quản lý vòng đời tài khoản, mã hóa bảo mật mật khẩu và cấp phát JWT.	8081
Product Service	Spring Boot + Spring Data JPA	Đảm nhiệm quản lý thông tin danh mục sản phẩm và trạng thái kho vật lý.	8082
Order Service	Spring Boot + Spring Data JPA	Xử lý logic đặt hàng, ghi nhận hóa đơn và phát hành sự kiện hệ thống.	8083

Notification Service	Spring Boot	Tiến trình ngàm kết nối nội bộ, thực thi tác vụ tiêu thụ tin nhắn và gửi email.	Nội bộ (No Port)
Message Broker	RabbitMQ (AMQP)	Hạ tầng trung gian quản lý các hàng đợi thông điệp bất đồng bộ.	15672 (UI Dashboard)
Database	PostgreSQL	Hệ quản trị cơ sở dữ liệu quan hệ lưu trữ dữ liệu bền vững.	5432

Nguyên tắc cô lập dữ liệu (Data Isolation): Nhóm triển khai một container hạ tầng PostgreSQL duy nhất nhưng thực hiện khởi tạo 3 cơ sở dữ liệu logic cô lập hoàn toàn: user_db, product_db và order_db. Mỗi microservice chỉ có đặc quyền kết nối và thao tác trên đúng cơ sở dữ liệu được chỉ định, cấm triệt để hành vi truy vấn chéo database

CHƯƠNG 3: MÔ TẢ CHI TIẾT CÁC MICROSERVICES

3.1. User Service (Cổng 8081)

Đóng vai trò là trung tâm quản lý danh tính (Identity Provider) trong toàn bộ hệ thống phân tán:

Mã hóa bảo mật: Tiếp nhận dữ liệu đăng ký từ client, thực hiện băm mật khẩu bằng thuật toán một chiều BCrypt trước khi ghi xuống đĩa cứng.

Cấp phát Token tập trung: Khi người dùng cung cấp đúng thông tin xác thực, dịch vụ sinh chuỗi JSON Web Token (JWT) ký số bảo mật, trả về phía client để làm chứng nhận truy cập.

Endpoint hỗ trợ Gateway: Cung cấp đường dẫn `/internal/auth/validate` cho phép API Gateway gọi kiểm tra nhanh chữ ký và tính hợp lệ của token theo thời gian thực.

Quản lý dữ liệu: Chịu trách nhiệm tương tác duy nhất với cơ sở dữ liệu `user_db`.

3.2. Product Service (Cổng 8082)

Đảm nhiệm vai trò quản lý tài nguyên sản phẩm và tính toán số dư kho hàng vật lý:

REST API công khai: Cung cấp các tài nguyên đọc dữ liệu phi bảo mật như xem danh sách sản phẩm hoặc chi tiết từng mặt hàng.

API nội bộ (Internal API): Cung cấp endpoint `/api/v1/products/{id}/check-inventory` phục vụ tác vụ kiểm tra đồng bộ lượng hàng còn lại từ Order Service gửi sang.

Xử lý bất đồng bộ: Đăng ký lắng nghe hàng đợi tin nhắn kết nối với RabbitMQ. Khi nhận được tín hiệu sự kiện đơn hàng được tạo lập, dịch vụ tự động trừ số lượng sản phẩm vật lý trong cơ sở dữ liệu.

Quản lý dữ liệu: Chịu trách nhiệm tương tác duy nhất với cơ sở dữ liệu `product_db`.

3.3. Order Service (Cổng 8083)

Là hạt nhân điều phối toàn bộ luồng nghiệp vụ mua sắm và giao dịch trên hệ thống:

Kiểm tra trạng thái đồng bộ: Khi nhận yêu cầu tạo đơn (Checkout), dịch vụ sử dụng HTTP Client gọi đồng bộ sang Product Service để kiểm tra độ sẵn sàng của hàng hóa.

Quản lý trạng thái đơn: Ghi nhận thông tin đơn hàng mới vào cơ sở dữ liệu với trạng thái ban đầu là PENDING (Chờ xử lý).

Kích hoạt sự kiện (Event Publishing): Đóng gói thông tin đơn hàng vừa khởi tạo thành một thông điệp chuẩn cấu trúc dữ liệu và bắn vào RabbitMQ Exchange dưới tên sự kiện order.created.

Quản lý dữ liệu: Chịu trách nhiệm tương tác duy nhất với cơ sở dữ liệu order_db.

3.4. Notification Service (Không mở cổng)

Là một dịch vụ thành phần chạy ngầm (Pure Consumer), tách biệt hoàn toàn khỏi các tác vụ tương tác trực tiếp với mạng ngoài:

Tiêu thụ thông điệp: Kết nối liên tục và lắng nghe trực tiếp luồng dữ liệu đổ về hàng đợi notification.queue trên RabbitMQ Broker.

Xử lý nghiệp vụ phụ: Trích xuất các trường dữ liệu cần thiết từ thông điệp đơn hàng (tên khách hàng, email, giá trị hóa đơn) để tiến hành in nhật ký (log) và giả lập luồng gửi email xác nhận giao dịch.

Tính bền vững: Có khả năng tự động quét và xử lý lại toàn bộ lượng tin nhắn còn tồn đọng trong queue ngay khi dịch vụ được kích hoạt khởi động lại sau sự cố.

CHƯƠNG 4: THIẾT KẾ RESTFUL API

Mọi API hệ thống được thiết kế nghiêm ngặt tuân thủ theo nguyên tắc định danh tài nguyên và sử dụng đúng các phương thức HTTP (HTTP Verbs) để thể hiện hành động xử lý dữ liệu.

4.1. User Service API

Toàn bộ các yêu cầu xác thực hệ thống đều được tập trung xử lý tại phân hệ này:

Phương thức	Endpoint	Mô tả	Xác thực
POST	/api/v1/auth/register	Tiếp nhận thông tin cấu trúc người dùng để đăng ký tài khoản mới	Không
POST	/api/v1/auth/login	Xác thực thông tin tài khoản, sinh và trả về chuỗi mã Token JWT	Không
GET	/api/v1/users/{id}	Truy xuất toàn bộ thông tin chi tiết hồ sơ cá nhân của một người dùng theo ID	JWT
PUT	/api/v1/users/{id}	Cập nhật thông tin sửa đổi của hồ sơ cá nhân	JWT
GET	/internal/auth/validate	Cổng API ẩn phục vụ việc kiểm tra độ tin cậy của mã	Nội bộ

		token từ API Gateway gọi sang (Chỉ mở nội bộ).	
--	--	--	--

4.2. Product Service API

Phân tách rõ ràng giữa các quyền đọc dữ liệu công khai và quyền chỉnh sửa hệ thống dữ liệu của quản trị viên:

Phương thức	Endpoint	Mô tả	Xác thực
GET	/api/v1/products	Trả về mảng danh sách toàn bộ sản phẩm đang được kinh doanh trên hệ thống	Không
GET	/api/v1/products/{id}	Trả về thông tin thuộc tính chi tiết của một mặt hàng cụ thể theo mã ID	Không
POST	/api/v1/products	Thêm mới một bản ghi sản phẩm vào hệ thống dữ liệu	JWT(Admin)

PUT	/api/v1/products/{id}	Chỉnh sửa các trường thông tin dữ liệu của một mặt hàng hiện có	JWT(Admin)
DELETE	/api/v1/products/{id}	Thực hiện xóa bỏ bản ghi hoặc ẩn sản phẩm khỏi hệ thống danh mục	JWT(Admin)
GET	/api/v1/products/{id}/check-inventory	API phục vụ tác vụ kiểm tra số lượng hàng tồn kho vật lý	Nội bộ

4.3. Order Service API

Quản lý luồng đặt hàng và theo dõi thông tin đơn hàng cá nhân của người dùng:

Phương thức	Endpoint	Mô tả	Xác thực
POST	/api/v1/orders	Khởi tạo quy trình mua hàng và thanh toán cho các mục hàng trong giỏ	JWT
GET	/api/v1/orders/{id}	Xem thông tin chi tiết hóa đơn tài chính của một đơn hàng cụ thể theo mã ID	JWT

GET	/api/v1/orders/my-orders	Trích xuất toàn bộ lịch sử danh sách đơn hàng đã từng phát sinh của tài khoản đang kết nối	JWT
PUT	/api/v1/orders/{id}/status	Điều chỉnh trạng thái xử lý của đơn hàng sang các bước tiếp theo trong vòng đời	JWT(Admin)

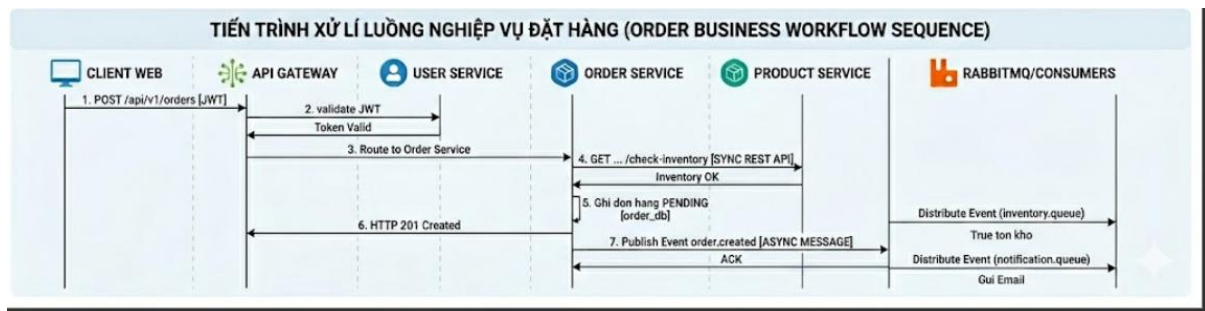
CHƯƠNG 5: LUỒNG MESSAGE QUEUE

5.1. Tổng quan cơ chế hướng sự kiện (Event-Driven Architecture)

Để giải quyết bài toán bề gãy sự phụ thuộc chặt chẽ giữa dịch vụ điều phối giao dịch (Order Service) và các phân hệ hậu cần, hệ thống áp dụng nền tảng tích hợp trung gian RabbitMQ làm Message Broker.

Hệ thống vận hành theo kiến trúc hướng sự kiện (Event-Driven Architecture) dựa trên mẫu thiết kế Publish/Subscribe. Tại thời điểm Order Service hoàn tất việc ghi nhận thông tin hóa đơn và khởi tạo đơn hàng thành công, dịch vụ này sẽ đóng vai trò là một Producer, phát hành một thông điệp sự kiện mang tên order.created và đẩy trực tiếp vào trung tâm điều phối. Luồng thông điệp này sẽ được phân phối đồng thời đến hai dịch vụ độc lập là Product Service và Notification Service nhằm thực hiện các tác vụ chuyên biệt theo từng miền nghiệp vụ cụ thể mà không làm tắc nghẽn luồng xử lý chính.

5.2. Đặc tả chi tiết tiến trình xử lý luồng nghiệp vụ đặt hàng (Sequence Workflow)



Hình ảnh 5.1: tiến trình xử lý luồng nghiệp vụ

Quy trình truyền thông tin và tương tác tuần tự giữa các thành phần dịch vụ từ thời điểm người dùng cuối phát động yêu cầu đặt hàng cho đến khi hoàn tất các tác vụ chạy ngầm được mô tả chi tiết qua các bước sau:

1. Gửi yêu cầu tạo đơn hàng (Step 1): Ứng dụng client (Client Web) phát động một HTTP Request sử dụng phương thức POST gửi tới điểm cuối /api/v1/orders. Yêu cầu này bắt buộc phải chèn chuỗi ký số JSON Web Token (JWT) vào tiêu đề (Header) nhằm mục đích định danh.

Xác thực và Điều phối (Step 2): Lá chắn bảo mật API Gateway đánh chặn gói tin, thực hiện giải mã và xác thực tính hợp lệ của Token JWT thông qua việc truy vấn

nội bộ tới User Service. Khi xác thực thành công, Gateway tiến hành định tuyến (Routing) request xuống dịch vụ xử lý giao dịch chuyên biệt là Order Service.

1. Kiểm tra dữ liệu tồn kho đồng bộ (Step 3): Để ngăn chặn hiện tượng bán quá số lượng thực tế (Overselling), Order Service kích hoạt luồng giao tiếp đồng bộ (Synchronous Communication via REST API), thực hiện gửi một HTTP GET Request tới endpoint `/products/{id}/check-inventory` thuộc Product Service. Product Service truy vấn dữ liệu thời gian thực và trả về trạng thái hợp lệ ("Còn hàng").
2. Lưu trữ trạng thái tạm thời (Step 4): Sau khi nhận tín hiệu đồng bộ hợp lệ từ phân hệ kho, Order Service tiến hành ghi nhận bản ghi đơn hàng mới vào cơ sở dữ liệu `order_db` với trạng thái vòng đời ban đầu được thiết lập là PENDING (Chờ xử lý).
3. Giải phóng tài nguyên Client (Step 5): Nhằm tối ưu hóa trải nghiệm người dùng và giảm thiểu độ trễ phản hồi (Response Time), Order Service ngay lập tức cắt đứt dòng đợi của Client bằng cách trả về mã trạng thái thành công HTTP 201 Created kèm thông báo "Đặt hàng thành công".
4. Phát hành sự kiện hệ thống (Step 6): Ngay sau khi phản hồi Client, Order Service đóng gói metadata của đơn hàng thành dữ liệu có cấu trúc (JSON Payload) và phát hành (Publish) sự kiện `order.created` lên cổng trung gian RabbitMQ Fanout Exchange.
5. Tiêu thụ thông điệp bất đồng bộ (Step 7): Do áp dụng mô hình định tuyến phát sóng, RabbitMQ Fanout Exchange tự động nhân bản thông điệp và phân phối đồng thời xuống các hàng đợi. Các tiến trình Consumer chạy ngầm tại hai dịch vụ sẽ tự động nhặt tin nhắn và xử lý song song:
 - Product Service: Tiêu thụ thông điệp từ hàng đợi kho để thực thi câu lệnh cập nhật giảm số lượng tồn kho vật lý của sản phẩm trong cơ sở dữ liệu `product_db`.
 - Notification Service: Tiêu thụ thông điệp từ hàng đợi thông báo để trích xuất thông tin khách hàng, kết nối với Mail Server ngoại biên và thực hiện quy trình gửi email hóa đơn điện tử cho khách hàng.

5.3. Thông số cấu hình kỹ thuật hạ tầng RabbitMQ

Để cam kết tính toàn vẹn dữ liệu, chống mất mát thông điệp khi xảy ra sự cố và tối ưu hóa hiệu năng điều phối, hạ tầng Message Broker được cấu hình nghiêm ngặt theo các thông số kỹ thuật trong bảng dưới đây:

Bảng 5.1: Đặc tả thông số cấu hình hạ tầng RabbitMQ Broker

Thuộc tính	Giá trị cấu hình	Mô tả ý nghĩa kỹ thuật và vai trò trong hệ thống
Exchange Name	order.exchange	Định danh cổng tiếp nhận tập trung cho toàn bộ các sự kiện phát sinh từ vòng đời của đơn hàng.
Exchange Type	fanout	Cơ chế định tuyến phát sóng (Broadcast). Mọi thông điệp gửi đến Exchange này sẽ được sao chép và đẩy ngay vào tất cả các hàng đợi liên kết mà không cần so khớp Routing Key, đảm bảo tốc độ xử lý tối đa.
Queue 1	inventory.queue	Hàng đợi chuyên biệt được gán kết (Binding) với Exchange, phục vụ riêng cho tiến trình lắng nghe và xử lý cập nhật kho vật lý của Product Service.
Queue 2	notification.queue	Hàng đợi chuyên biệt được gán kết (Binding) với Exchange, phục vụ riêng cho tiến trình tiêu thụ dữ liệu và kích hoạt luồng gửi mail của Notification Service.

Durability	Durable	Thuộc tính bền vững hệ thống. Cấu hình này bắt buộc RabbitMQ Broker phải liên tục ghi nhận trạng thái và thông điệp của hàng đợi xuống phân vùng ổ đĩa cứng vật lý của container, đảm bảo dữ liệu không bị bốc hơi nếu hệ thống gặp sự cố sập nguồn phân cứng đột ngột.
Manual ACK	Bật (True)	Cơ chế xác nhận thủ công. Hệ thống tắt bỏ tính năng tự động xóa tin nhắn. Các Consumer tại Product Service và Notification Service chỉ được gửi tín hiệu ACK quay lại Broker khi và chỉ khi đã hoàn tất trọn vẹn việc cập nhật Database hoặc gửi mail thành công, ngăn chặn mất mát tin nhắn khi hệ thống bị crash giữa chừng.

CHƯƠNG 6 : THỬ NGHIỆM VÀ ĐÁNH GIÁ

6.1. Môi trường thử nghiệm

Toàn bộ hệ thống E-Commerce Microservices được thiết kế để có thể khởi chạy và kiểm thử hoàn chỉnh trên môi trường cục bộ chỉ với một lệnh duy nhất. Để đạt được điều đó, nhóm áp dụng Docker làm nền tảng đóng gói và Docker Compose làm công cụ điều phối, đảm bảo mọi thành viên trong nhóm cũng như hội đồng chấm đều có thể tái tạo chính xác môi trường vận hành mà không phụ thuộc vào cấu hình máy tính cá nhân.

Cấu hình môi trường thử nghiệm chi tiết:

- Hệ điều hành nền tảng: Windows 11 tích hợp Docker Engine và Docker Compose v2.x.
- Kiến trúc mạng nội bộ (Docker Network): Các container được cô lập và giao tiếp với nhau qua mạng ảo nội bộ. Nhằm đảm bảo bảo mật, API Gateway (chạy tại cổng 8080) là điểm giao tiếp duy nhất mở ra bên ngoài để tiếp nhận yêu cầu từ ứng dụng Frontend (chạy tại cổng 3000). Không có Microservice nào (User, Product, Order, Notification) được phép truy cập trực tiếp từ bên ngoài.
- Hệ sinh thái dịch vụ hạ tầng:
 - PostgreSQL: Hệ quản trị cơ sở dữ liệu quan hệ chính. Để thể hiện tính độc lập dữ liệu giữa các service, hệ thống khởi tạo 3 database logic riêng biệt trên cùng một container: user_db, product_db và order_db. Mỗi service chỉ được cấp quyền truy cập đúng database của mình.
 - RabbitMQ: Đóng vai trò Message Broker trung tâm cho kiến trúc Event-Driven, sử dụng cơ chế Fanout Exchange để phân phối sự kiện đồng thời tới nhiều service tiêu thụ. Giao diện quản trị RabbitMQ Management UI được mở tại cổng 15672 phục vụ theo dõi và kiểm thử trực quan.

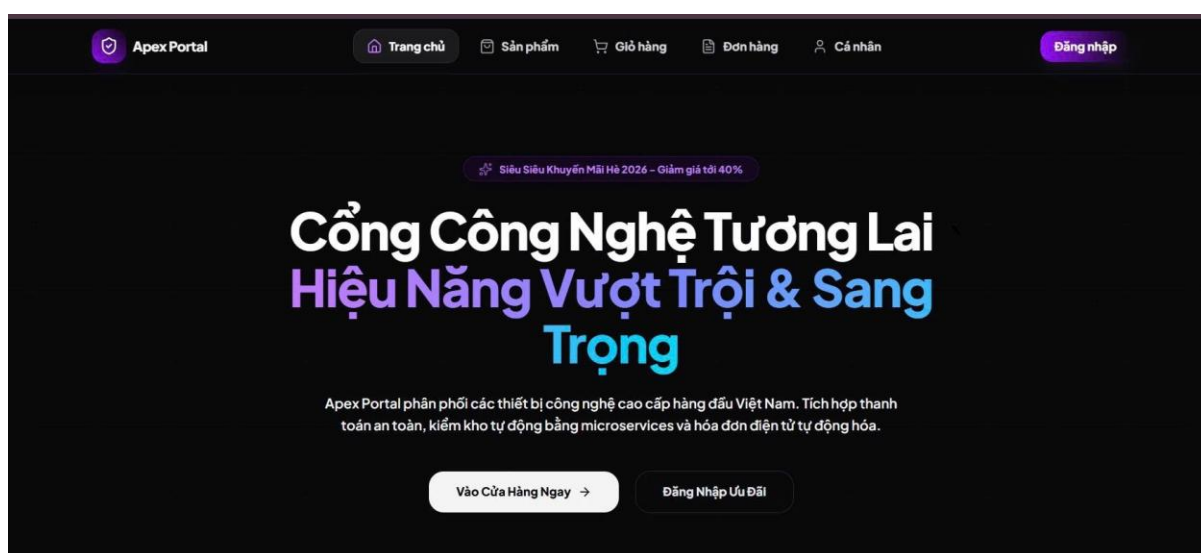
- Spring Cloud Gateway: Đóng vai trò API Gateway, xử lý xác thực JWT tập trung và định tuyến request tới đúng service nội bộ tương ứng.

Quá trình khởi động toàn bộ cụm hệ thống bao gồm Frontend, API Gateway, 4 Microservice lõi và các dịch vụ hạ tầng được tự động hóa

Toàn bộ 8 container (api-gateway, user-service, product-service, order-service, notification-service, frontend-web, postgres-db, rabbitmq) sẽ được khởi tạo, kết nối mạng nội bộ và sẵn sàng phục vụ mà không cần cấu hình thủ công thêm bất kỳ bước nào.

6.2. Kịch bản thử nghiệm

Để xác minh tính đúng đắn của các luồng nghiệp vụ liên dịch vụ, khả năng bảo mật của API Gateway cũng như tính chịu lỗi của kiến trúc Event-Driven, nhóm đã thiết lập và thực thi hệ thống kiểm thử :



Hình ảnh 6.1 : Giao diện trang chủ

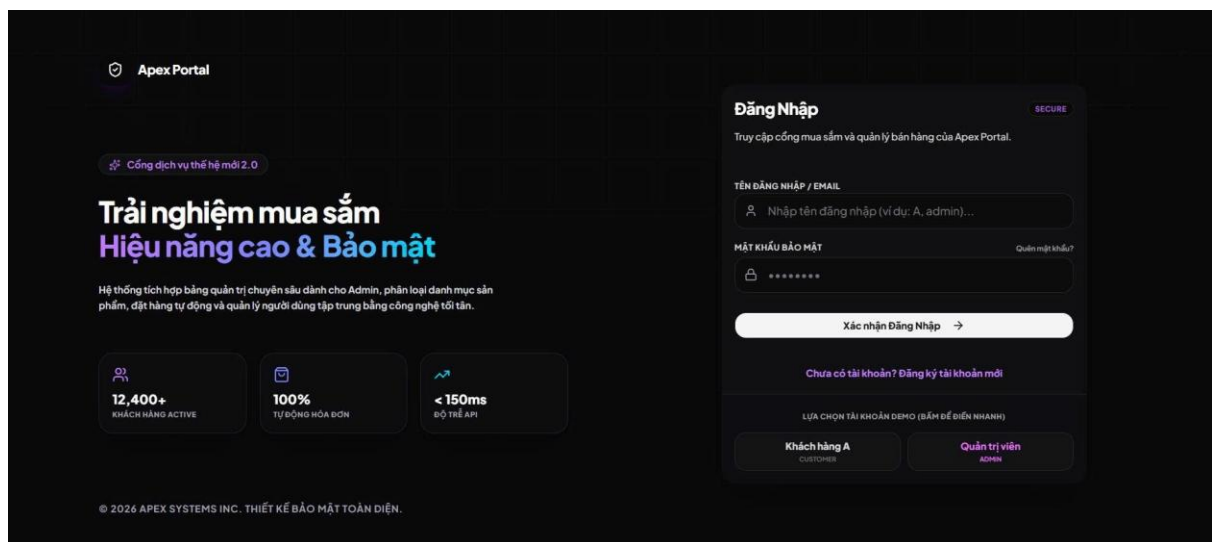
Kịch bản 1 : Luồng đặt hàng chuẩn chỉnh (Happy Path)

- Mục tiêu: Xác mục luồng nghiệp vụ mua hàng thành công khi tài khoản hợp lệ và sản phẩm còn đủ số lượng tồn kho.
- Điều kiện tiên quyết: Khách hàng đã có tài khoản, sản phẩm ID PROD-001 đang có số lượng tồn kho là 50.

Bước 1: Đăng nhập vào hệ thống

Khách hàng truy cập giao diện Frontend tại `http://localhost:3000`, điều hướng đến trang Đăng nhập và nhập thông tin xác thực hợp lệ. Hệ thống gửi request `POST /api/v1/auth/login` đến API Gateway, Gateway định tuyến xuống User Service để xác thực thông tin tài khoản.

Kết quả đạt được: User Service xác thực thành công, sinh chuỗi JSON Web Token (JWT) và trả về phía client. Giao diện chuyển hướng vào trang chủ, hiển thị thông tin tài khoản đã đăng nhập ở góc trên bên phải. JWT được lưu trữ tại phía client và tự động đính kèm vào header của mọi request tiếp theo.

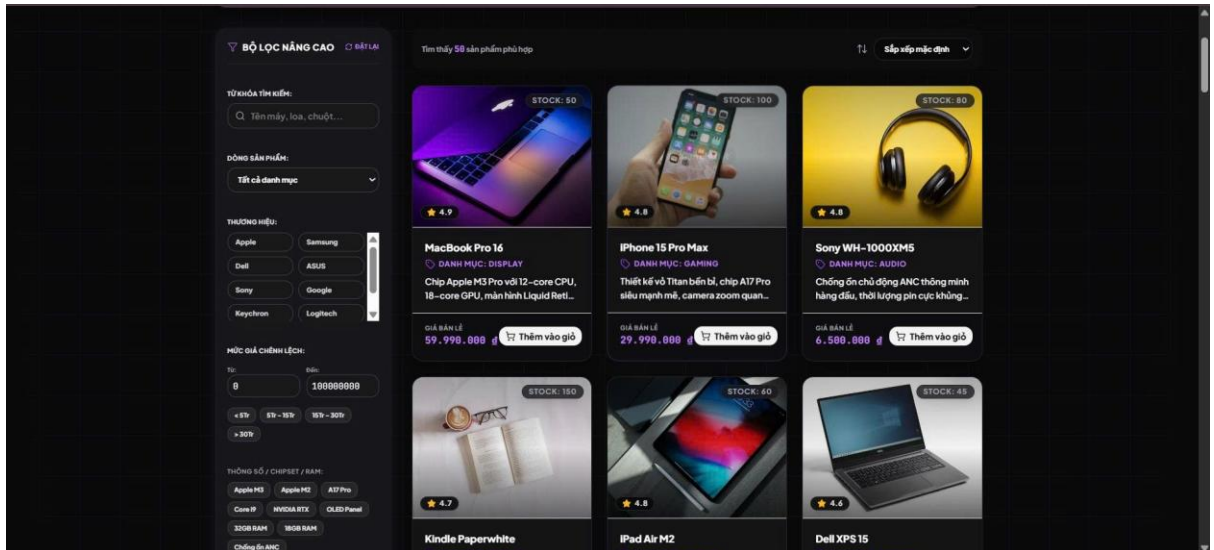


Hình ảnh 6.2 : Giao diện đăng nhập

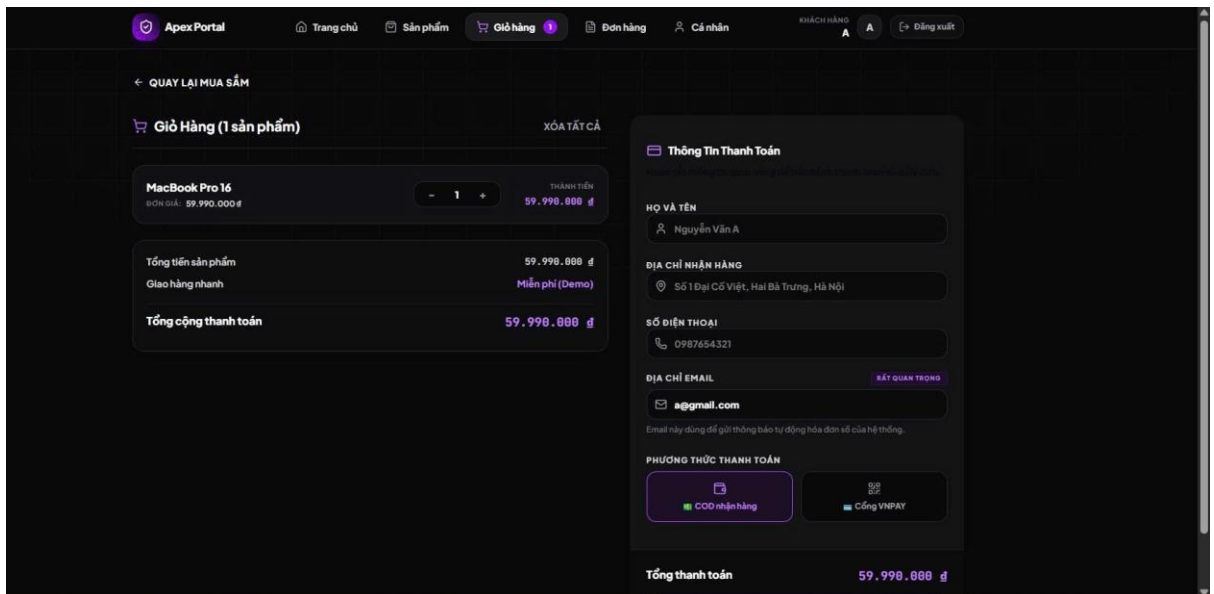
Bước 2: Bấm nút "Đặt hàng" trên giao diện

Khách hàng điều hướng đến trang Sản phẩm. Hệ thống gọi `GET /api/v1/products` để tải danh sách hàng hóa. Khách hàng chọn sản phẩm MacBook Pro 16, xem thông tin chi tiết và nhấn nút **"Thêm vào giỏ hàng"**, sau đó tiến hành **Checkout**.

Kết quả đạt được: Giao diện hiển thị đầy đủ danh sách sản phẩm kèm thông tin tồn kho thời gian thực. Giỏ hàng cập nhật chính xác số lượng và tổng tiền. Trang thanh toán hiển thị đầy đủ thông tin khách hàng và địa chỉ giao hàng.



Hình ảnh 6.3 : giao diện sản phẩm



Hình ảnh 6.4 : giao diện hiển thị thanh toán

Bước 3: Kiểm tra đơn hàng chi tiết

Khách hàng kiểm tra thông tin đơn hàng trên trang thanh toán (họ tên, địa chỉ nhận hàng, email, phương thức thanh toán) và nhấn nút "Đặt hàng". Client gửi request POST /api/v1/orders kèm JWT Token đến API Gateway.

Phía sau, hệ thống thực hiện tuần tự các bước sau:

- API Gateway xác thực JWT hợp lệ qua User Service, định tuyến request xuống Order Service.
- Order Service gọi đồng bộ sang Product Service để kiểm tra tồn kho - nhận phản hồi "Còn hàng".
- Order Service ghi nhận đơn hàng vào order_db với trạng thái PENDING.
- Order Service trả phản hồi HTTP 201 về client ngay lập tức.
- Order Service publish sự kiện order.created vào RabbitMQ Fanout Exchange.
- Product Service và Notification Service tiêu thụ message song song: trừ tồn kho và gửi email xác nhận.

Kết quả đạt được: Giao diện hiển thị thông báo "Đặt hàng thành công" ngay lập tức mà không có độ trễ chờ xử lý email. Đơn hàng mới xuất hiện trong trang Lịch sử đơn hàng với trạng thái PENDING. Số lượng tồn kho của PROD-001 trong product_db giảm đúng số lượng đã đặt. Log của Notification Service ghi nhận thành công việc gửi email xác nhận đến địa chỉ email của khách hàng.

CHI TIẾT ĐƠN HÀNG Đóng

ĐƠN HÀNG #27

THÔNG TIN KHÁCH HÀNG

TÊN KHÁCH HÀNG	ĐỊA CHỈ EMAIL
Nhu Quỳnh	ntng92@gmail.com
SỐ ĐIỆN THOẠI	ĐỊA CHỈ NHẬN HÀNG
0339579602	Phùng Khoang Hà Nội

DANH SÁCH MUA SẴM

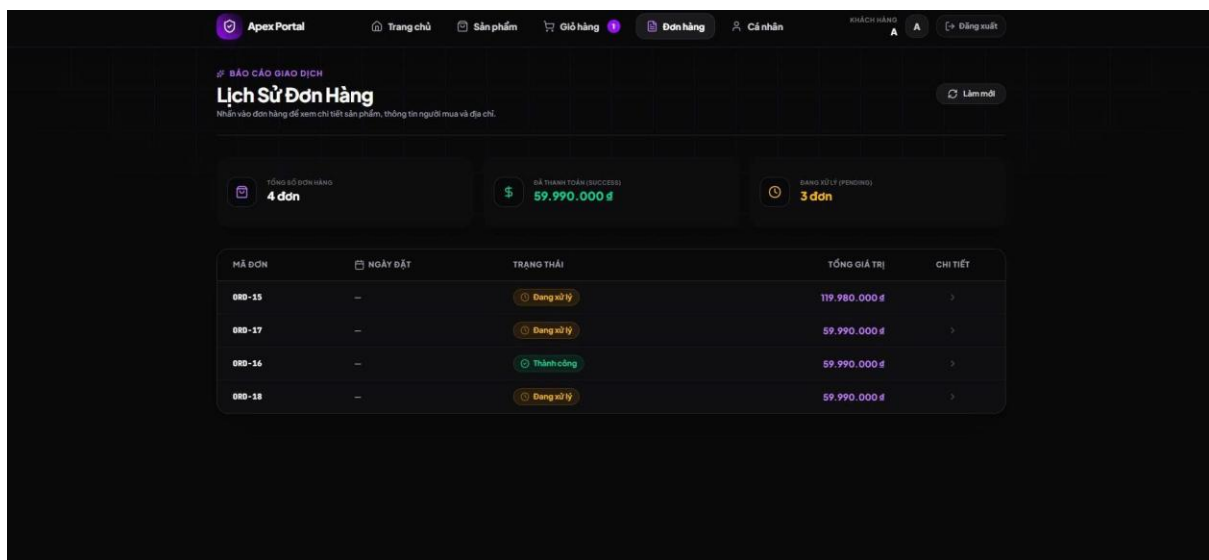
Sản phẩm #1578	14.990.000 đ
SỐ LƯỢNG: 1	
Sản phẩm #1579	45.990.000 đ
SỐ LƯỢNG: 1	
TỔNG CỘNG ĐƠN HÀNG	60.980.000 đ

TRẠNG THÁI: **PENDING** Duyệt đơn hàng Hủy đơn hàng

Hình ảnh 6.5 : Giao diện chi tiết đơn hàng

Khách hàng điều hướng đến trang Đơn hàng và gọi GET /api/v1/orders/my-orders để xem toàn bộ lịch sử giao dịch.

Kết quả đạt được: Hệ thống trả về danh sách đầy đủ các đơn hàng đã phát sinh của tài khoản, bao gồm mã đơn hàng, ngày đặt, tổng giá trị và trạng thái xử lý hiện tại. Đơn hàng vừa tạo hiển thị đúng thông tin và trạng thái PENDING.



Hình ảnh 6.6 : Lịch sử đơn hàng

Kịch bản 2: Luồng quản lý

Mục tiêu: Xác minh toàn bộ chức năng quản trị của hệ thống hoạt động đúng đắn bao gồm giám sát và duyệt đơn hàng, quản lý danh mục sản phẩm và quản lý tài khoản người dùng. Đồng thời xác minh cơ chế phân quyền tại API Gateway chặn hoàn toàn các tác vụ quản trị từ tài khoản không có quyền Admin.

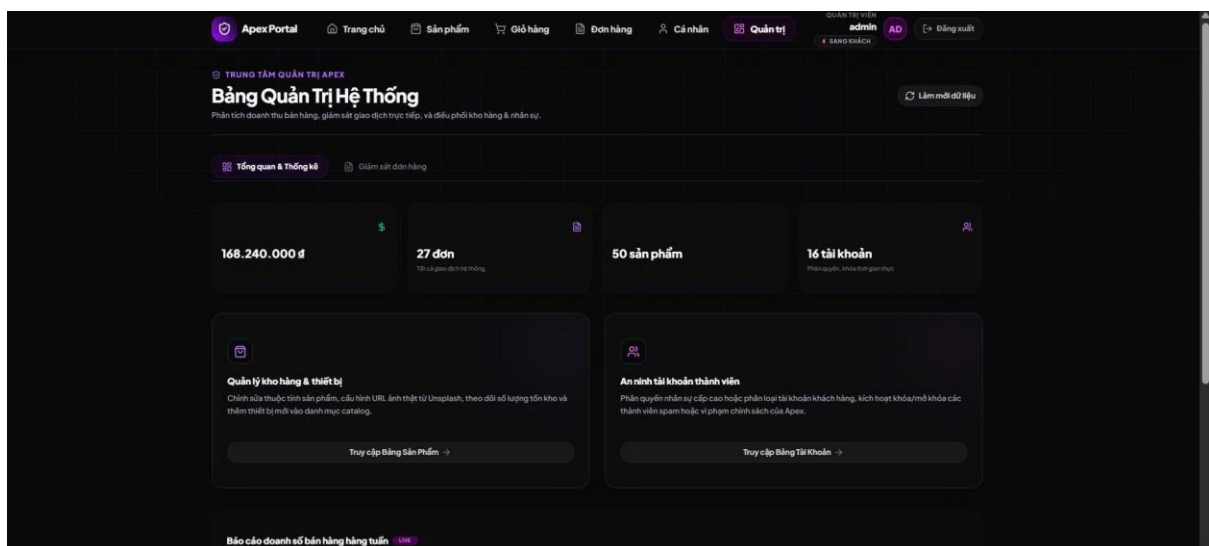
Điều kiện tiên quyết:

- Tài khoản Admin đã được khởi tạo sẵn trong hệ thống với role ADMIN.
- Hệ thống đang có sẵn dữ liệu đơn hàng và sản phẩm từ kịch bản 1.

Bước 1: Đăng nhập trang quản trị hệ thống

Admin truy cập giao diện Frontend, điều hướng đến trang Đăng nhập và xác thực bằng tài khoản Admin. User Service xác thực thành công và sinh JWT có những thông tin role ADMIN bên trong payload.

Kết quả đạt được: Sau khi đăng nhập, giao diện hiển thị thêm mục Quản trị trên thanh điều hướng - mục này hoàn toàn ẩn đối với tài khoản khách hàng thông thường. Bảng điều khiển (Dashboard) hiển thị tổng quan hệ thống: tổng doanh thu, tổng số đơn hàng, tổng số sản phẩm và tổng số tài khoản đang hoạt động.



Hình ảnh 6.7 : Giao diện trang quản trị hệ thống

Bước 2: Giám sát và duyệt đơn hàng

Admin điều hướng đến trang Giám sát đơn hàng, hệ thống tải toàn bộ danh sách đơn hàng của tất cả khách hàng trên hệ thống. Admin xem chi tiết đơn hàng vừa được tạo ở Kịch bản 1 và thực hiện duyệt đơn bằng cách gọi PUT /api/v1/orders/{id}/status để chuyển trạng thái từ PENDING sang CONFIRMED.

Kết quả đạt được: API Gateway kiểm tra JWT của Admin, xác nhận role ADMIN hợp lệ và cho phép request đi qua. Order Service cập nhật trạng thái đơn hàng thành công trong order_db. Giao diện hiển thị trạng thái mới ngay lập tức. Nếu thử thực hiện thao tác tương tự bằng tài khoản khách hàng thông thường, Gateway lập tức từ chối với mã lỗi HTTP 403 Forbidden.

Mã Đơn	Email Người Mua	Ngày Đặt	Tổng Tiền	Trạng Thái	Thao Tác Duyệt
27	ntng0720@gmail.com	2024-05-27T09:43:39.045387	60.990.000 đ	PENDING	Đuyệt Đơn / Hủy Đơn
26	ntng0720@gmail.com	27/5/2024	29.990.000 đ	SUCCESS	Đuyệt Đơn / Hủy Đơn
25	ntng0720@gmail.com	27/5/2024	890.000 đ	FAILED	Đuyệt Đơn / Hủy Đơn
24	ntng0720@gmail.com	27/5/2024	5.490.000 đ	SUCCESS	Đuyệt Đơn / Hủy Đơn
23	admin@apex.com	27/5/2024	3.290.000 đ	FAILED	Đuyệt Đơn / Hủy Đơn
22	admin@apex.com	27/5/2024	1.590.000 đ	FAILED	Đuyệt Đơn / Hủy Đơn
21	ntng0720@gmail.com	27/5/2024	72.770.000 đ	SUCCESS	Đuyệt Đơn / Hủy Đơn
20	hahongminh723@gmail.com	27/5/2024	3.500.000 đ	FAILED	Đuyệt Đơn / Hủy Đơn
19	hahongminh723@gmail.com	27/5/2024	59.990.000 đ	PENDING	Đuyệt Đơn / Hủy Đơn
18	nguyensana@gmail.com	27/5/2024	59.990.000 đ	PENDING	Đuyệt Đơn / Hủy Đơn

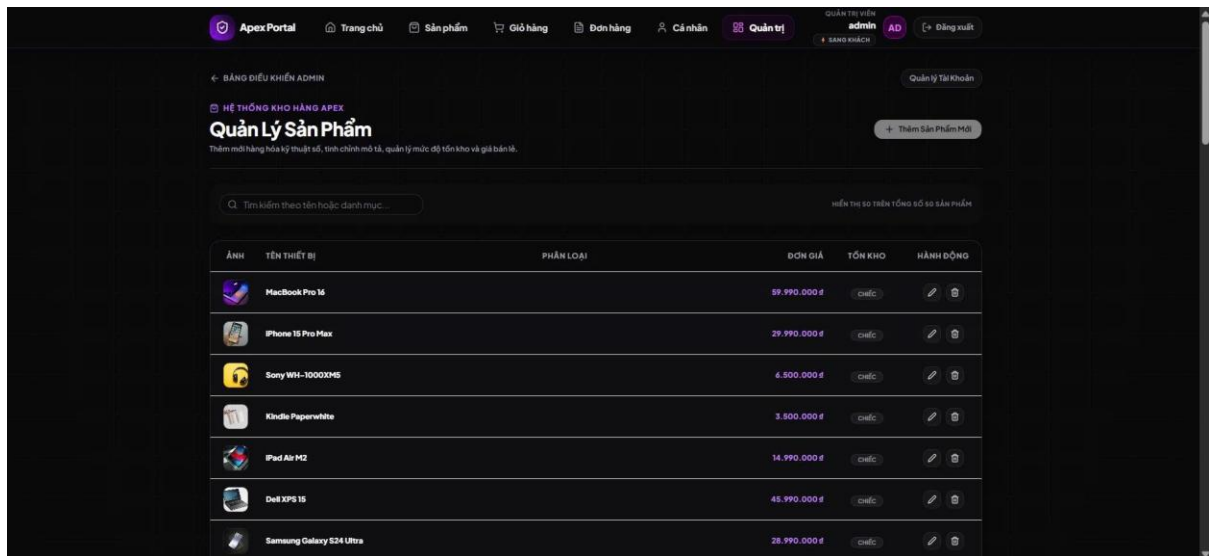
Hình ảnh 6.8 : Giao diện giám sát đơn hàng

Bước 3: Quản lý sản phẩm

Admin điều hướng đến trang Quản lý Sản phẩm và thực hiện lần lượt 3 tác vụ:

- Thêm sản phẩm mới: Gọi POST `/api/v1/products` với đầy đủ thông tin tên, mô tả, giá, số lượng tồn kho và hình ảnh.
- Cập nhật thông tin: Chọn một sản phẩm hiện có, chỉnh sửa giá bán và gọi PUT `/api/v1/products/{id}`.
- Xóa sản phẩm: Chọn một sản phẩm ngừng kinh doanh và gọi DELETE `/api/v1/products/{id}`.

Kết quả đạt được: Cả 3 tác vụ đều yêu cầu JWT có role ADMIN — Gateway thực thi bộ lọc phân quyền chính xác. Sản phẩm mới xuất hiện ngay trên trang danh sách sau khi thêm. Thông tin cập nhật phản ánh đúng trên giao diện khách hàng. Sản phẩm bị xóa không còn hiển thị trong danh mục công khai. Mọi thay đổi được ghi nhận chính xác và bền vững trong `product_db`.

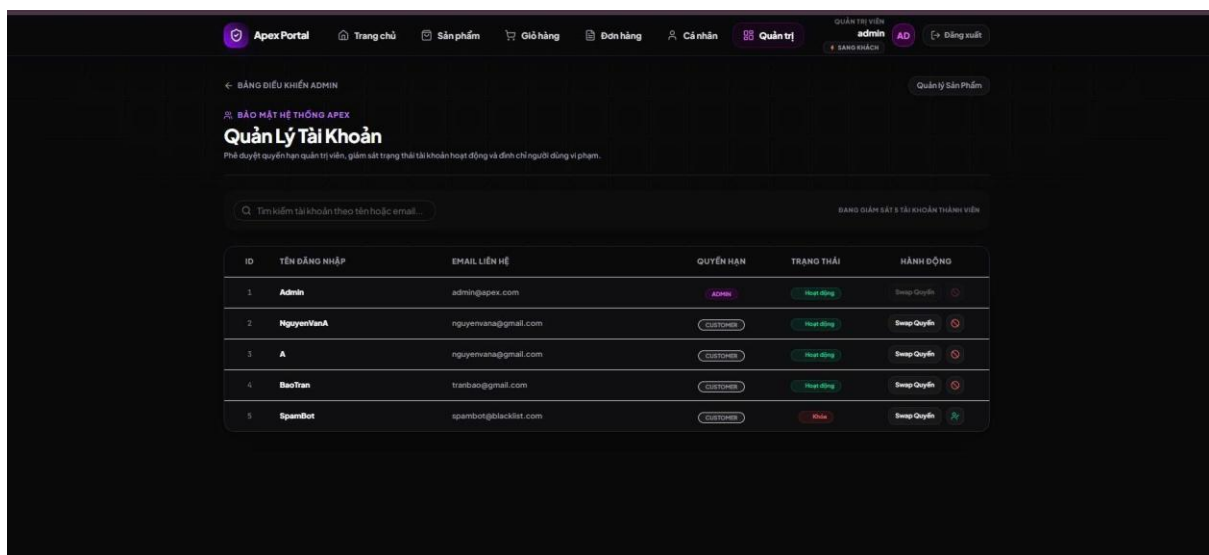


Hình ảnh 6.9 : Giao diện quản lý sản phẩm

Bước 4: Quản lý người dùng

Admin điều hướng đến trang Quản lý Tài khoản, xem danh sách toàn bộ người dùng đã đăng ký trên hệ thống bao gồm thông tin tên đăng nhập, email, role và trạng thái hoạt động.

Kết quả đạt được: Hệ thống hiển thị đầy đủ danh sách tài khoản với phân loại rõ ràng giữa role ADMIN và CUSTOMER. Admin có thể tra cứu, lọc tài khoản theo email hoặc tên. Mọi thao tác truy xuất dữ liệu người dùng đều được kiểm soát chặt chẽ — chỉ tài khoản mang role ADMIN mới được phép truy cập endpoint quản trị này.



Hình ảnh 6.10 : Giao diện quản lý tài khoản

Kịch bản 3: Đặt hàng khi sản phẩm hết hàng

Mục tiêu: Xác minh hệ thống từ chối đơn hàng và thông báo rõ ràng khi tồn kho không đủ, ngăn chặn tình trạng bán quá số lượng (Overselling).

Điều kiện tiên quyết: Sản phẩm ID PROD-002 có số lượng tồn kho bằng 0.

Bước 1: Đăng nhập bằng tài khoản khách hàng hợp lệ.

Bước 2: Truy cập trang sản phẩm PROD-002 và thực hiện đặt hàng.

Bước 3: Hệ thống phản hồi mã lỗi HTTP 400 Bad Request kèm thông báo "Sản phẩm đã hết hàng, vui lòng thử lại sau". Không có bản ghi nào được tạo trong `order_db`.

Kết quả đạt được: Order Service gọi đồng bộ sang Product Service, nhận trạng thái hết hàng, từ chối tạo đơn và không publish bất kỳ event nào vào RabbitMQ. Tính toàn vẹn dữ liệu được đảm bảo.

Kịch bản 4: Xác thực bảo mật — Token JWT không hợp lệ

- **Mục tiêu:** Xác minh API Gateway từ chối toàn bộ các request không có hoặc sai Token JWT, bảo vệ các endpoint nhạy cảm khỏi truy cập trái phép.
- **Công cụ:** Postman.

Bước 1: Gửi request POST `/api/v1/orders` qua API Gateway mà không đính kèm header Authorization.

Kết quả: Hệ thống trả về HTTP 401 Unauthorized ngay tại tầng Gateway, request không được chuyển tiếp xuống Order Service.

Bước 2: Gửi lại request với một chuỗi JWT giả mạo (sai chữ ký số).

Kết quả: Gateway phát hiện chữ ký không khớp qua endpoint `/internal/auth/validate` của User Service, trả về HTTP 401. Các Microservice nội bộ hoàn toàn không nhận được request này.

Kịch bản 5: Kiểm thử tính chịu lỗi — Fault Tolerance Demo

- **Mục tiêu:** Chứng minh tính tách rời (Decoupling) của kiến trúc Event-Driven. Khi Notification Service gặp sự cố, luồng đặt hàng chính và cập nhật tồn kho vẫn hoạt động bình thường, message không bị mất.

Bước 1: Giả lập sự cố: Tắt hoàn toàn Notification Service

Bước 2:Tạo tải: Thực hiện đặt liên tiếp 3 đơn hàng trên giao diện Frontend. Quan sát thấy giao diện vẫn phản hồi thông báo "Đặt hàng thành công" ngay lập tức, không có hiện tượng treo hoặc lỗi. Dữ liệu 3 đơn hàng được ghi nhận chính xác trong order_db với trạng thái PENDING. Tồn kho trong product_db được trừ đúng số lượng bởi Product Service.

Bước3:Kiểm tra hàng đợi: Truy cập RabbitMQ Management UI tại <http://localhost:15672>. Quan sát tab Queues, xác nhận notification.queue đang lưu trữ an toàn 3 message tồn đọng — không có message nào bị mất.

Bước 4 : Phục hồi hệ thống: Khởi động lại Notification Service:

Kết quả: Ngay khi container khởi động thành công, Notification Service tự động kết nối lại vào RabbitMQ Broker, tiêu thụ sạch 3 message tồn đọng và in log xác nhận xử lý gửi email cho từng đơn hàng ra màn hình console.

Kết luận kịch bản: Kiến trúc Message Queue đảm bảo tính bền vững dữ liệu (Durability) và tính chịu lỗi tuyệt đối — Order Service không hề hay biết Notification Service đang gặp sự cố, toàn bộ nghiệp vụ cốt lõi không bị ảnh hưởng.

6.3. Đánh giá tiêu chí

Dựa trên quá trình xây dựng và thử nghiệm thực tế, nhóm tự đánh giá mức độ hoàn thành theo từng tiêu chí đề ra như sau:

Tiêu chí	Yêu cầu	Kết quả	Mô tả chi tiết
Ít nhất 3 microservice	Bắt buộc	ĐẠT	Hệ thống có 4 service độc lập: User Service, Product Service, Order Service và Notification Service. Mỗi service chạy trên container riêng, triển khai và khởi động độc lập nhau.
RESTful API hoạt động	Bắt buộc	ĐẠT	Đầy đủ các phương thức GET, POST, PUT, DELETE. Đúng nguyên tắc thiết kế tài nguyên REST, trả về HTTP status code chuẩn. Đã kiểm thử toàn bộ endpoint qua Postman.
Message Queue	Bắt buộc	ĐẠT	Tích hợp RabbitMQ với Fanout Exchange. Luồng đặt hàng publish sự kiện order.created được cả Product Service (trừ tồn kho) và Notification Service (gửi email) tiêu thụ đồng thời.
Database độc lập	Bắt buộc	ĐẠT	3 database logic riêng biệt: user_db, product_db, order_db. Mỗi service chỉ được cấp quyền truy cập đúng database của mình, không service nào truy cập chéo dữ liệu của service khác.
Xử lý lỗi cơ bản	Bắt buộc	ĐẠT	Xử lý timeout khi gọi REST liên service, Dead Letter Queue cho message lỗi, retry tối đa 3 lần, ghi log đầy đủ ở cấp độ INFO/ERROR bằng SLF4J + Logback.

Docker Compose	Khuyến khích	ĐẠT	Toàn bộ 8 container được khởi động chỉ bằng lệnh docker-compose up --build -d. Không cần cấu hình thủ công thêm bất kỳ bước nào.
API Gateway	Khuyến khích	ĐẠT	Spring Cloud Gateway đóng vai trò điểm vào duy nhất, xử lý định tuyến tập trung và xác thực JWT trước khi chuyển tiếp request vào các service nội bộ.
JWT Authentication	Khuyến khích	ĐẠT	Xác thực tập trung tại API Gateway. JWT được cấp phát bởi User Service, giải mã và kiểm tra tại Gateway. Các service nội bộ không cần tự xác thực lại.

6.4. Nhận xét ưu điểm và hạn chế

6.4.1. Ưu điểm

Về kiến trúc và thiết kế:

Tính độc lập cao: Mỗi microservice được triển khai trong container riêng biệt, có thể khởi động, dừng và cập nhật độc lập mà không ảnh hưởng đến các service còn lại. Điều này được minh chứng trực tiếp qua Kịch bản 5 khi Notification Service bị tắt hoàn toàn nhưng luồng đặt hàng chính vẫn vận hành bình thường.

Tính tách rời dữ liệu: Việc áp dụng mô hình Database-per-service với 3 database logic riêng biệt (user_db, product_db, order_db) đảm bảo không có sự phụ thuộc dữ liệu chéo giữa các service, tạo nền tảng vững chắc cho việc mở rộng độc lập từng thành phần trong tương lai.

Bảo mật tập trung: Cơ chế xác thực JWT tập trung tại API Gateway giúp loại bỏ sự trùng lặp logic xác thực ở từng service, đồng thời tạo ra một điểm kiểm soát bảo mật duy nhất, nhất quán và dễ bảo trì.

Giao tiếp lai hiệu quả: Sự kết hợp giữa REST API đồng bộ (kiểm tra tồn kho)

và RabbitMQ bắt đồng bộ (trừ kho, gửi email) tối ưu hóa cả tốc độ phản hồi cho người dùng lẫn tính chịu lỗi của hệ thống - khách hàng nhận được phản hồi ngay lập tức mà không cần chờ các tác vụ hậu cần hoàn tất.

Triển khai đơn giản: Toàn bộ hệ thống gồm 8 container được khởi động chỉ bằng một lệnh `docker-compose up --build -d`, giúp việc tái tạo môi trường trở nên nhanh chóng và nhất quán trên mọi máy tính.

Tính bền vững dữ liệu: Cấu hình Durable Queue và Manual ACK trên RabbitMQ đảm bảo không có message nào bị mất ngay cả khi hệ thống gặp sự cố đột ngột, tăng cường độ tin cậy của toàn bộ luồng nghiệp vụ.

Về trải nghiệm người dùng:

Giao diện Frontend được xây dựng trên Next.js phản hồi nhanh, hiển thị thông tin tồn kho thời gian thực và cung cấp đầy đủ chức năng cho cả hai nhóm đối tượng Khách hàng và Quản trị viên.

Phân quyền RBAC được thực thi minh bạch - giao diện tự động hiện/ẩn chức năng quản trị dựa trên role của token, đảm bảo trải nghiệm phù hợp với từng nhóm người dùng.

6.4.2. Hạn chế

Chưa xử lý giao dịch phân tán (Distributed Transaction): Trong trường hợp Order Service tạo đơn thành công nhưng Product Service xử lý trừ tồn kho thất bại liên tục và message chuyển vào DLQ, hệ thống chưa có cơ chế tự động rollback trạng thái đơn hàng. Giải pháp lý tưởng là triển khai Saga Pattern để điều phối giao dịch phân tán một cách nhất quán.

Chưa có Circuit Breaker: Khi Product Service phản hồi chậm hoặc liên tục timeout, Order Service hiện tại sẽ chờ đợi và không có cơ chế ngắt mạch tự động. Việc tích hợp Resilience4j Circuit Breaker sẽ giúp hệ thống phát hiện sự cố sớm và chuyển sang chế độ fallback, tránh hiệu ứng domino làm sập toàn bộ luồng xử lý.

Gửi email chưa thực tế: Notification Service hiện tại chủ yếu giả lập luồng gửi email bằng cách in log ra console thay vì kết nối với Mail Server thực tế. Để hoàn thiện, cần tích hợp dịch vụ SMTP thực như Gmail SMTP hoặc SendGrid.

Chưa có Centralized Logging: Mỗi service hiện tại ghi log riêng lẻ trong container của mình. Việc tích hợp ELK Stack (Elasticsearch, Logstash, Kibana) sẽ giúp tập trung toàn bộ log về một nơi, hỗ trợ tìm kiếm, phân tích và cảnh báo lỗi hiệu quả hơn nhiều.

Chưa có Service Discovery động: Hiện tại địa chỉ các service được cấu hình tĩnh trong Docker Compose. Trong môi trường production với nhiều instance, cần tích hợp Eureka Service Discovery hoặc Consul để các service tự tìm thấy nhau một cách linh hoạt.

Chưa triển khai trên Cloud: Hệ thống hiện chỉ chạy trên môi trường cục bộ (localhost). Việc triển khai lên Google Cloud Platform hoặc Kubernetes sẽ mang lại khả năng mở rộng theo chiều ngang (horizontal scaling) và tính sẵn sàng cao (high availability) trong môi trường thực tế.

Bên cạnh những kết quả tích cực đã đạt được, nhóm cũng nhận thức rõ những hạn chế còn tồn tại như chưa xử lý được giao dịch phân tán hoàn chỉnh, chưa có circuit breaker và centralized logging. Đây là những hướng phát triển cụ thể và có giá trị mà nhóm định hướng tiếp tục nghiên cứu trong giai đoạn tiếp theo, nhằm đưa hệ thống tiệm cận hơn với tiêu chuẩn của một hệ thống phân tán sản xuất thực tế.

TÀI LIỆU THAM KHẢO

- [1] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [2] C. Richardson, Microservices Patterns: With Examples in Java. Shelter Island, NY, USA: Manning Publications, 2018.
- [3] I. C. Walls, Spring in Action, 6th ed. Shelter Island, NY, USA: Manning Publications, 2022.
- [4] Spring Cloud, "Spring Cloud Gateway Documentation," Spring Framework, 2025.[Online]Available: <https://docs.spring.io/spring-cloud-gateway/docs/current/reference/html/>. [Accessed: May. 2025].
- [5] Spring Framework, "Spring Boot Reference Documentation," VMware Tanzu, 2025. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. [Accessed: May. 2025].
- [6] RabbitMQ, "RabbitMQ Official Documentation," Broadcom Inc., 2025. [Online]. Available: <https://www.rabbitmq.com/documentation.html>. [Accessed: May. 2025].
- [7] Docker Inc., "Docker Compose Documentation," Docker Inc., 2025. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: May. 2025].
- [8] PostgreSQL Global Development Group, "PostgreSQL Documentation," PostgreSQL, 2025. [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: May. 2025].
- [9] Auth0, "Introduction to JSON Web Tokens," Auth0 Inc., 2025. [Online]. Available: <https://jwt.io/introduction>. [Accessed: May. 2025].